

M12x — Cell-by-cell logical inspection (18 cells)

Experiment ID: M12x (Milestone 1 Part 2 expanded)

Status: Stage-3 qualitative inspection **done** / **analysed** (18/18 written)

Date started: 2026-07-20

Parent record: M1.2-expanded.md

Artefacts: causal/outputs/aba_learning/grid/M12x_{ecai2024,aamas2025}/cells/{dgp}__target-{t}/

Side-car matrix: causal/outputs/aba_learning/grid/M12x_summary.md

Per-cell detector reports: causal/outputs/aba_learning/grid/M12x_cell_reports/

Fixtures: causal/experiments/handcrafted_m12x.py

Catalogue: docs/research/milestone_plans/milestone1_part2/m12x_units_reference.tex

Grid run: 2026-07-20 (18/18 cells solved; summary regenerated same day)

This document inspects each of the 18 (**arm** $\times$ **fixture** $\times$ **target**) cells. For every completed cell it records: the locked setup and $\mathcal{H}_t^*$; a clear reading of how ABA Learning produced the learned framework; and whether and why that framework matches or fails the intended hypothesis and semantic success description.

This is **not** Russo-style Causal ABA. The object of study is unguided ABA Learning recovery of mechanism-aligned target rules from categorical tabular background knowledge and positive/negative examples.

Reading conventions. Learned δ is the new intensional material in `bk.sol.aba` after variable-canonical background knowledge subtraction. Exact match means the learned target rules equal $\mathcal{H}_t^*$ after normalisation. ASP `pos_covered` / `neg_rejected` fractions are coverage facts from the side-car; they are not a substitute for intensional match. Acceptance remains inspection-first against the Approach semantic description.

1 Inspection checklist (18)

#	Arm	Fixture	Target	Exact $\mathcal{H}^*$?	Short verdict
1	ECAI	m12_u1_separator_copy	x_2	N	Assumption-mediated via isolated x_0 ; parent only in contraries
2	ECAI	m12_u2_collider_min	x_2	N	Elegant assumption-mediated both-parents-nonzero (not the locked 4-clause form)
3	ECAI	m12_u3_collider_max	x_2	N	x_0 -case OR: pure unary on $x_0 \in \{1,2\}$; assumption only on $x_0 = 0$ slice
4	ECAI	m12_u4_fork_asymmetric	x_1	Y	Exact $\mathcal{H}^*$: two unary parent rules; sibling unused
5	ECAI	m12_u4_fork_asymmetric	x_2	Y	Exact $\mathcal{H}^*$: two unary parent rules; sibling unused (mirror of cell 4)
6	ECAI	m12_u5_chain_curated	x_2	N	Ancestor x_0 case + assumptions; parent only in contraries

#	Arm	Fixture	Target	Exact $\mathcal{H}^*$?	Short verdict
7	ECAI	m12_u6_g1_min_diff	x_2	N	Assumption-mediated both-parents-nonzero (as U2); descendant unused
8	ECAI	m12_u6_g1_min_diff	x_3	N	Ancestor x_0 + assumptions; x_2 unused; not $x_1 > x_2$ form
9	ECAI	m12_u7_diamond_noisy	x_3	N	Covering failure: x_0 +assumptions; x_2 never selected; ASP neg 0/3
10	AAMAS	m12_u1_separator_copy	x_2	N	Six specialised conjunctions; isolated x_0 retained beside parent
11	AAMAS	m12_u2_collider_min	x_2	Y	Exact $\mathcal{H}^*$: four both-parent conjunctions, no assumptions
12	AAMAS	m12_u3_collider_max	x_2	N	Eight pairwise positive-cell conjunctions; not the locked unary OR
13	AAMAS	m12_u4_fork_asymmetric	x_1	N	Parent + sibling conjunctions; cites distractor x_2
14	AAMAS	m12_u4_fork_asymmetric	x_2	N	Parent + sibling conjunctions; cites distractor x_1
15	AAMAS	m12_u5_chain_curated	x_2	N	Parent + grandparent conjunctions; ancestor retained
16	AAMAS	m12_u6_g1_min_diff	x_2	N	Both-parent rules + descendant x_3 ; cites_descendant
17	AAMAS	m12_u6_g1_min_diff	x_3	N	$\mathcal{H}^*$ parent pairs + ancestor x_0 ; cites_non_parent_ancestor
18	AAMAS	m12_u7_diamond_noisy	x_3	N	$\mathcal{H}^*$ parent pairs + ancestor x_0 ; cites_non_parent_ancestor

2 Shared regime (locked)

- Alphabet $K = \{0, 1, 2\}$; nonzero-positive labels $E^+ = \{t \neq 0\}$, $E^- = \{t = 0\}$ unless a unit card overrides.
- Background knowledge: exact-value predicates `xi_val_v` for every non-target column (descendants kept as distractors).
- Configs: ECAI (`configs/ecai2024_config.pl`); AAMAS (`configs/aamas2025_config.pl`).
- Reference hypotheses $\mathcal{H}_t^*$ and structural sets are as in the unit catalogue and `m12x_summary.py` look-up tables.

Effective options (confirmed in cell traces):

- **ECAI:** brave learning; `folding_mode(nd)`; `folding_selection(any)`; `folding_space(all)`; `asm_intro(relto)`; `post_folding_test_ entailment(true)`.
- **AAMAS:** brave learning; greedy folding; published AAMAS options consulted verbatim from `configs/aamas2025_config.pl`.

2.1 Shared ABALearn control flow (gen)

Both M12x arms run the same generalisation procedure in `gen.pl`. What differs between ECAI and AAMAS is how each fold is computed and which non-intensional rule is selected for folding; those differences are stated at the top of each batch. The shared skeleton is as follows.

Learning begins from background knowledge together with a list of **non-intensional** (rote) rules. Rote learning introduces one such rule per positive example, of the form $t(A) \leftarrow A = i$, and later may introduce further rote rules for contraries of newly introduced assumptions. Only

non-intensional rules are candidates for folding. Background **val** predicates are folding *material*: they may appear in folded bodies, but they are not themselves selected as rules to fold.

The outer driver is **genT**. It first attempts a complete derivation with the current folding settings via **gen1**. Under non-deterministic folding only, if that attempt fails entirely, **genT** increases the folding-token budget by one (up to **folding_steps**) and retries. Under greedy folding that deepening branch does not apply. Procedural success for **genT** means that **gen1** returned a finished framework; it does not by itself certify that the written solution separates $\langle E^+, E^- \rangle$ under the project's post-hoc ASP coverage check.

gen1 selects one foldable non-intensional rule S , computes a fold F of S , and passes F to **gen2**. If no foldable non-intensional rule remains, the trace reports **gen1: nothing to fold** and **gen1** succeeds, returning the current ABA framework as the learned solution.

gen2 tests whether extending the current framework by F already entails $\langle E^+, E^- \rangle$. If it does, the fold is kept (**extended ABA entails $\langle E^+, E^- \rangle$ - using folding**) and control returns to **gen1**. If it does not, and assumption introduction is enabled, **gen2** attempts repair rather than treating the failed entailment as an immediate global failure. Under the shared M12x setting **asm_intro(relto)**, it first looks for an existing assumption relative to the body of F ; if one is found, control passes to **gen3**. Otherwise it mints a new assumption α , forms the patched rule with α in the body, and continues through **gen4** into **gen6**.

gen3 handles reuse of an existing assumption under **relto**. If the framework with the reused assumption entails $\langle E^+, E^- \rangle$, the trace reports **OK** and control returns to **gen1**. If not, it reports **KO** and that alternative fails, allowing Prolog to backtrack into other folding choices for the same non-intensional rule where available.

gen4 / **gen5** only route after a new assumption has been generated. Both M12x arms use **relto**, so **gen4** proceeds directly to **gen6**. The **sechk** path (**gen5**) is unused in this experiment.

gen6 rote-learns contrary atoms for the new assumption (typically rules of the form $c_\alpha(A) \leftarrow A = j$) and appends them to the non-intensional list, so they become later folding targets. Under **cautious** learning, **gen6** then re-checks entailment and fails if the check does not hold. Under **brave** learning—which both M12x arms use—that re-check is skipped, and control returns to **gen1** even if $\langle E^+, E^- \rangle$ is not yet entailed.

Two outcome distinctions used throughout the cell write-ups follow from this skeleton.

First, **engine termination** (**nothing to fold**, writing **bk.sol.***) means the non-intensional list was emptied by a successful **gen1** derivation. It does not mean that every preferred parent appeared in a body, nor that tokens were raised to the configured ceiling.

Second, the grid runner's **solved** label records that ABA Learning completed and produced a solution artefact. **ASP coverage** (positives covered / negatives rejected under brave entailment on **bk.sol.asp**) is a separate, post-hoc check of whether that artefact separates $\langle E^+, E^- \rangle$. A cell may therefore be runner-solved and still fail to reject negatives.

3 Batch A — ECAI (M12x_ecai2024)

3.1 ECAI folding control flow

Batch A uses `configs/ecai2024_config.pl`: `learning_mode(brave)`, `folding_mode(nd)`, `folding_steps(10)`, `folding_selection(any)`, `folding_space(all)`, and `asm_intro(relto)`. The shared **gen** skeleton above still applies; what follows is how folding and selection specialise that skeleton on this arm.

Which rule is folded. Under `folding_selection(any)`, `gen1` may select any remaining non-intensional rule (after deleting those already subsumed). There is no preference for “most general” rote rules.

How a fold is computed. Under `folding_mode(nd)`, each fold call is token-bounded. The token counter starts at 1 every run. With k tokens, non-deterministic folding may replace body literals by background (or, under `folding_space(all)`, other foldable) rule heads up to k times in that call. The first success of the fold procedure is often a **one-literal** body when $k = 1$: the sample identifier $A = i$ is replaced by a single `val` predicate, the trace reports `0: DONE`, and that candidate is passed to `gen2`. Alternative fold literals for the same rote rule remain as Prolog choice points, but they are explored only if the accepted path later fails.

When the token budget rises. The configured ceiling `folding_steps(10)` is not the starting budget. `genT` increases tokens by one (and prints `* Increasing folding tokens to: ...`) only if the whole `gen1` search at the current budget fails. If assumption introduction keeps producing a continuing derivation that eventually reaches `nothing to fold`, tokens never rise. Multi-literal parent-parent bodies therefore appear only when earlier one-token (or low-token) repair paths fail hard enough to force deepening.

What happens when a fold overgeneralises. This is the distinctive ECAI behaviour relative to AAMAS. If `gen2` finds that the folded rule does not entail $\langle E^+, E^- \rangle$, the engine does not treat that as a cue to discard the fold and search for a better body as its primary next step. Under `asm_intro(relto)` it attempts assumption introduction on **that same** fold: reuse an existing relative assumption (`gen3: OK/KO`) or mint a new α and rote-learn its contrary (`gen6`). Under brave mode, `gen6` does not require entailment before returning to `gen1`. The effective procedure is therefore **cover-and-repair**: accept a locally available fold, and if it overgeneralises, push exceptions into assumptions and contraries rather than retracting the fold in favour of another intension.

`folding_space(all)`. Folding is not restricted to the original background-knowledge block; fold candidates may include other rules admitted into the folding table. In the M12x tables the first successful one-literal folds are still typically `xi_val_*` feature predicates, ordered by how `select_rule` enumerates matching heads.

How to read ECAI traces in this batch. Lines of the form `begin nd folding / 1: folding ... with x*val* / 0: DONE` mark a token-bounded fold. `extended ABA does not entail ... looking for assumption followed by generating NEW assumption or found: alpha_... OK/KO` mark the repair path. Absence of `Increasing folding tokens to:` means the derivation completed at the starting budget of one token. As in the shared skeleton, `nothing to fold` and runner `solved` record procedural completion; ASP coverage must still be checked separately.

3.2 1. ECAI $\times$ m12_u1_separator_copy $\times$ $t = x_2$

Cell directory: M12x_ecai2024/cells/m12_u1_separator_copy__target-x2/

Runner outcome: solved

ASP coverage: positives 6/6, negatives 3/3

3.2.1 Setup

The graph is a separator $x_1 \rightarrow x_2$ with an isolated variable x_0 . The mechanism sets x_2 equal to x_1 on the full nine-row factorial over (x_0, x_1) . Under nonzero-positive labelling the positive examples are rows $\{2, 3, 5, 6, 8, 9\}$ and the negative examples are rows $\{1, 4, 7\}$. Background

knowledge contains exact-value predicates for x_0 and x_1 ; the target column x_2 is excluded. The ancestors of x_2 are $\{x_1\}$; the isolated variable x_0 is a distractor; there are no descendants. The design role of this unit is to test whether the learner recovers a unary parent rule and avoids citing the isolated distractor.

3.2.2 Intended hypothesis $\mathcal{H}_{x_2}^*$

```
x2(A) :- x1_val_1(A).
x2(A) :- x1_val_2(A).
```

These two clauses characterise $x_2 \neq 0$ by the parent x_1 being nonzero. They must not cite the isolated distractor x_0 .

3.2.3 What happened

This cell uses the ECAI published configuration on unit U1 with learning target x_2 . The learner returned a solved framework with full ASP coverage of the examples. That framework does not match $\mathcal{H}_{x_2}^*$ intensionally. The target rules cite the isolated distractor x_0 together with assumptions, and the true parent x_1 appears only in the contrary conditions.

3.2.4 What the learner did

Learning begins from rote rules that cover each positive example by its sample identifier:

```
x2(A) :- A=2.
x2(A) :- A=3.
x2(A) :- A=5.
x2(A) :- A=6.
x2(A) :- A=8.
x2(A) :- A=9.
```

Under non-deterministic folding with selection **any** and folding space **all**, the first successful fold for a representative positive in each x_0 -slice replaces that identifier by a value predicate of the isolated column x_0 . The first fold yields a body containing only `x0_val_0`; later folds yield only `x0_val_1` and only `x0_val_2`:

```
x2(A) :- x0_val_0(A).    % from A=2; overgeneralises within the x0=0 slice
x2(A) :- x0_val_1(A).    % from A=5
x2(A) :- x0_val_2(A).    % from A=8
```

Each of those bodies overgeneralises within its slice, because the negative example that shares the same x_0 value would also be covered. Entailment of the labelled example set therefore fails, and assumption introduction relative to the failing body adds a fresh assumption `alpha_1`, `alpha_2`, or `alpha_3`. The remaining positive examples in the same slice are then deleted as subsumed. The repaired target rules are:

```
x2(A) :- alpha_1(A), x0_val_0(A).
x2(A) :- alpha_2(A), x0_val_1(A).
x2(A) :- alpha_3(A), x0_val_2(A).
```

Contrary learning is driven by the three negative examples, which are exactly the rows with $x_1 = 0$. For each contrary, folding first tries the matching x_0 literal already used in the corresponding

target rule. That attempt is rejected, because the engine cannot introduce an assumption already tied to that body. Folding then succeeds with `x1_val_0`:

```
c_alpha_1(A) :- x1_val_0(A).
c_alpha_2(A) :- x1_val_0(A).
c_alpha_3(A) :- x1_val_0(A).
```

The accepted solution therefore has three assumption-mediated target rules keyed by x_0 , and three identical contrary rules that block those assumptions whenever $x_1 = 0$. Body-scope variables of the target rules are $\{x_0\}$. The parent x_1 appears only in the framework-scope contrary conditions.

3.2.5 Match assessment

The learned target rules are not an exact match to $\mathcal{H}_{x_2}^*$.

On the complete partition of the table by x_0 , an assumption remains undefeated on a row in that slice if and only if `x1_val_0` fails, which is exactly when $x_1 \in \{1, 2\}$. Taken together, the three rules therefore cover the six positives and reject the three negatives. In that limited extensional sense the solution agrees with the nonzero-positive copy semantics of U1.

Intensionally the solution is misaligned with the intended hypothesis. The learner folded the isolated distractor x_0 before the true parent x_1 . It repaired the resulting overgeneralisation by introducing assumptions. It recovered the true separator only as contrary conditions on x_1 . The intended hypothesis cites x_1 directly in ordinary target rules and does not mention x_0 at all.

Short verdict. The cell is solved and extensionally adequate, but intensionally it encodes the parent concept through assumptions keyed by the isolated distractor, with the true parent appearing only in the contraries.

3.3 2. ECAI $\times$ m12_u2_collider_min $\times$ $t = x_2$

Cell directory: M12x_ecai2024/cells/m12_u2_collider_min__target-x2/

Runner outcome: solved

ASP coverage: positives 4/4, negatives 5/5

3.3.1 Setup

Unit U2 is a collider $x_0 \rightarrow x_2 \leftarrow x_1$ with mechanism $x_2 := \min(x_0, x_1)$ on the full nine-row factorial over (x_0, x_1) . Under nonzero-positive labelling, $x_2 \neq 0$ if and only if both parents are nonzero. The positive examples are therefore rows $\{5, 6, 8, 9\}$ and the negative examples are rows $\{1, 2, 3, 4, 7\}$. Background knowledge contains exact-value predicates for both parents; the target x_2 is excluded. There are no sibling or descendant distractors beyond the two parents themselves. The design role of this unit is to test recovery of a conjunctive both-parent rule. The locked reference $\mathcal{H}_{x_2}^*$ enumerates that concept as four ordinary `val` conjunctions, one for each nonzero pair of parent values.

3.3.2 Intended hypothesis $\mathcal{H}_{x_2}^*$

```
x2(A) :- x0_val_1(A), x1_val_1(A).
x2(A) :- x0_val_1(A), x1_val_2(A).
```

```
x2(A) :- x0_val_2(A), x1_val_1(A).
x2(A) :- x0_val_2(A), x1_val_2(A).
```

3.3.3 What happened

This cell uses the ECAI published configuration on unit U2 with learning target x_2 . The learner returned a solved framework with full ASP coverage. It did not emit the four locked case clauses. Instead it recovered a compact assumption-mediated representation of the same semantic idea: x_0 is nonzero in the target bodies, and x_1 is nonzero through undefeated assumptions whose contraries fire exactly when $x_1 = 0$.

3.3.4 What the learner did

Learning begins from rote rules for the four positives:

```
x2(A) :- A=5.
x2(A) :- A=6.
x2(A) :- A=8.
x2(A) :- A=9.
```

Under non-deterministic folding with selection **any** and folding space **all**, the first fold for $A = 5$ (where $x_0 = 1$) replaces the sample identifier by `x0_val_1` alone:

```
x2(A) :- x0_val_1(A).
```

That body overgeneralises: it would also cover the negative row $A = 4$, where $x_0 = 1$ but $x_1 = 0$. Entailment of the labelled example set therefore fails, and assumption introduction adds `alpha_1`:

```
x2(A) :- alpha_1(A), x0_val_1(A).
```

The remaining positive in the same $x_0 = 1$ slice ($A = 6$) is then deleted as subsumed. The same pattern occurs for the $x_0 = 2$ slice: fold $A = 8$ to `x0_val_2`, introduce `alpha_2`, and delete $A = 9$ as subsumed:

```
x2(A) :- alpha_2(A), x0_val_2(A).
```

Contrary learning is driven by the negatives that sit in those same x_0 slices, namely $A = 4$ and $A = 7$ (both have $x_1 = 0$). Folding each contrary first tries the matching x_0 literal already used in the corresponding target rule; that attempt is rejected. Folding then succeeds with `x1_val_0`:

```
c_alpha_1(A) :- x1_val_0(A).
c_alpha_2(A) :- x1_val_0(A).
```

In the ASP solution these assumptions are realised by default-negation rules scoped to the same x_0 -slices:

```
alpha_1(A) :- not c_alpha_1(A), x0_val_1(A).
alpha_2(A) :- not c_alpha_2(A), x0_val_2(A).
```

Those clauses are what allow each assumption to hold on its slice precisely when the corresponding contrary fails, that is when $x_1 \neq 0$. Without them, `alpha_1` and `alpha_2` would never be

derived, and the target rules that depend on them would not cover the positives.

Negatives with $x_0 = 0$ (rows $\{1, 2, 3\}$) never enter these target bodies, so they need no contrary repair. Body-scope variables of the target rules are $\{x_0\}$. The second parent x_1 appears in the contrary conditions, and thereby controls whether the assumptions succeed.

3.3.5 Match assessment

Relative to the locked case enumeration $\mathcal{H}_{x_2}^*$, this is not an exact syntactic match. The learner does not produce the four ordinary both-parent conjunctions.

Judged against the semantic success description for U2, however, the learned framework is a strong and arguably more elegant intensional solution. It characterises nonzero x_2 under min by combining an explicit nonzero condition on one parent in the target bodies with an assumption-mediated nonzero condition on the other parent. On an $x_0 \in \{1, 2\}$ row, x_2 holds if and only if the assumption is undefeated if and only if $x_1 \neq 0$. That is exactly “both parents are nonzero,” expressed with two target rules and the ABA assumption machinery rather than with four case clauses. ASP coverage is complete for that reason.

So the mismatch with $\mathcal{H}_{x_2}^*$ should not be read as a failure to recover the intended concept. It is a mismatch with one particular syntactic presentation of that concept. The learned form uses assumptions to represent nonzeroness compactly.

Short verdict. Not an exact match to the locked four-clause $\mathcal{H}_{x_2}^*$, but an elegant assumption-mediated encoding of the same both-parents-nonzero concept, with full coverage.

3.4 3. ECAI $\times$ m12_u3_collider_max $\times$ $t = x_2$

Cell directory: M12x_ecai2024/cells/m12_u3_collider_max__target-x2/

Runner outcome: solved

ASP coverage: positives 8/8, negatives 1/1

3.4.1 Setup

Unit U3 is the disjunctive counterpart of U2: the same collider $x_0 \rightarrow x_2 \leftarrow x_1$, now with mechanism $x_2 := \max(x_0, x_1)$ on the nine-row factorial. Under nonzero-positive labelling, $x_2 \neq 0$ if and only if at least one parent is nonzero. The positive examples are therefore rows $\{2, 3, 4, 5, 6, 7, 8, 9\}$ and the sole negative is row $\{1\}$. Background knowledge contains exact-value predicates for both parents; the target x_2 is excluded. The locked reference $\mathcal{H}_{x_2}^*$ is the four-clause `val` expansion of “at least one parent is nonzero.”

3.4.2 Intended hypothesis $\mathcal{H}_{x_2}^*$

```
x2(A) :- x0_val_1(A).
x2(A) :- x0_val_2(A).
x2(A) :- x1_val_1(A).
x2(A) :- x1_val_2(A).
```


3.4.3 What happened

This cell uses the ECAI published configuration on unit U3 with learning target x_2 . After the initial optimum there are eight rote rules, one for each positive. Non-deterministic folding proceeds by x_0 -slices and yields a mixed solution: pure unary rules for $x_0 \in \{1, 2\}$, and an assumption-mediated rule only on the incomplete $x_0 = 0$ slice. The learner returns a solved framework with full ASP coverage. The result is not an exact syntactic match to $\mathcal{H}_{x_2}^*$, but it is a compact case analysis that still expresses the intended disjunctive concept.

3.4.4 What the learner did

Learning begins from rote coverage of the eight positives. The first fold chosen is $A = 2$ (where $x_0 = 0, x_1 = 1$), which replaces the identifier by `x0_val_0` alone:

```
x2(A) :- x0_val_0(A).
```

That body would also cover the unique negative $A = 1$ (where $x_0 = 0$ and $x_1 = 0$), so entailment fails and assumption introduction adds `alpha_1`:

```
x2(A) :- alpha_1(A), x0_val_0(A).
```

The other positive in the same slice ($A = 3$) is then deleted as subsumed. Next, $A = 4$ folds to `x0_val_1`. Under max, every row with $x_0 = 1$ is already positive, so this one-literal rule entails $\langle E^+, E^- \rangle$ immediately and needs no assumption:

```
x2(A) :- x0_val_1(A).
```

Positives $A = 5$ and $A = 6$ are subsumed. Likewise $A = 7$ folds to a safe unary rule on the $x_0 = 2$ slice:

```
x2(A) :- x0_val_2(A).
```

Positives $A = 8$ and $A = 9$ are subsumed. Contrary learning for the negative $A = 1$ first tries `x0_val_0` (rejected), then succeeds with `x1_val_0`:

```
c_alpha_1(A) :- x1_val_0(A).
```

In the ASP solution the assumption is realised by

```
alpha_1(A) :- not c_alpha_1(A), x0_val_0(A).
```

so on the $x_0 = 0$ slice the assumption holds precisely when $x_1 \neq 0$. Body-scope variables of the target rules are $\{x_0\}$; x_1 appears only through the contrary / assumption pair.

3.4.5 Match assessment

This is not an exact match to $\mathcal{H}_{x_2}^*$. The learner does not emit the four unary parent rules of the locked OR expansion.

Extensionally, the learned theory is

$$(x_0 = 0 \wedge x_1 \neq 0) \vee (x_0 = 1) \vee (x_0 = 2),$$

which is exactly $x_0 \neq 0$ or $x_1 \neq 0$, the nonzero-positive semantics of max. ASP coverage is complete for that reason.

Intensionally this is an x_0 -first case analysis: the complete nonzero x_0 slices become ordinary unary rules, and only the incomplete $x_0 = 0$ slice needs an assumption to encode “ x_1 nonzero.” Relative to $\mathcal{H}_{x_2}^*$, the presentation differs (no free-standing `x1_val_1` / `x1_val_2` target rules), but the recovered concept is the intended disjunction. The pattern is related to ECAI on U2: assumption machinery is used where a one-literal fold would overgeneralise, while folds that are already safe are kept without assumptions.

Short verdict. Not an exact match to the locked four-rule OR, but a solved, fully covering x_0 -case analysis that encodes the same “at least one parent nonzero” concept, with assumptions only on the $x_0 = 0$ slice.

3.5 4. ECAI $\times$ m12_u4_fork_asymmetric $\times$ $t = x_1$

Cell directory: M12x_ecai2024/cells/m12_u4_fork_asymmetric__target-x1/

Runner outcome: solved

ASP coverage: positives 2/2, negatives 1/1

3.5.1 Setup

Unit U4 is a fork $x_0 \rightarrow x_1$, $x_0 \rightarrow x_2$ on the three-row source factorial over $x_0 \in \{0, 1, 2\}$. The asymmetric maps are $x_1 := 2$ if $x_0 \neq 2$ else 0, and $x_2 := 2$ if $x_0 \neq 0$ else 0. For target x_1 , nonzero-positive labelling gives $E^+ = \{1, 2\}$ and $E^- = \{3\}$; on the table, $x_1 \neq 0$ if and only if $x_0 \in \{0, 1\}$. Background knowledge includes exact-value predicates for the parent x_0 and the sibling distractor x_2 ; the target x_1 is excluded. The sibling is correlated with x_1 via x_0 but imperfect as a separator (`x2_val_2` holds on both positive row 2 and negative row 3). The locked reference $\mathcal{H}_{x_1}^*$ is the two-clause parent expansion of $x_0 \neq 2$.

3.5.2 Intended hypothesis $\mathcal{H}_{x_1}^*$

```
x1(A) :- x0_val_0(A).
x1(A) :- x0_val_1(A).
```

3.5.3 What happened

This cell uses the ECAI published configuration on unit U4 with learning target x_1 . After the initial optimum there are two rote rules, one for each positive. Non-deterministic folding replaces each identifier by the corresponding parent value in a single step, and both folds entail $\langle E^+, E^- \rangle$ immediately. The learner returns a solved framework with full ASP coverage, no assumptions, and no citation of the sibling.

3.5.4 What the learner did

Learning begins from rote coverage of the two positives:

```
x1(A) :- A=1.
x1(A) :- A=2.
```

The first fold chosen is $A = 1$ (where $x_0 = 0$), which replaces the identifier by `x0_val_0`:

```
x1(A) :- x0_val_0(A).
```

That body covers only the positive row with $x_0 = 0$ and does not cover the negative ($x_0 = 2$), so entailment succeeds without assumption introduction. The second fold is $A = 2$ (where $x_0 = 1$), which likewise yields a safe unary parent rule:

```
x1(A) :- x0_val_1(A).
```

Learning then stops with **nothing to fold**. Although sibling predicates (`x2_val_0`, `x2_val_2`) are present in the background knowledge, neither appears in a target body. Body-scope variables of the target rules are $\{x_0\}$.

3.5.5 Match assessment

This is an exact match to $\mathcal{H}_{x_1}^*$.

The two unary parent rules are precisely the locked `val` expansion of “ $x_0 \neq 2$ ” under nonzero-positive labelling of x_1 . ASP coverage is complete, the solution uses no assumptions, and the sibling distractor is not cited. Relative to the unit’s semantic success description, the learned framework characterises nonzero x_1 via the true parent without rote sample-id casework.

Short verdict. Exact match to $\mathcal{H}_{x_1}^*$: two unary parent rules, no assumptions, sibling unused, full coverage.

3.6 5. ECAI $\times$ m12_u4_fork_asymmetric $\times$ $t = x_2$

Cell directory: M12x_ecai2024/cells/m12_u4_fork_asymmetric__target-x2/

Runner outcome: solved

ASP coverage: positives 2/2, negatives 1/1

3.6.1 Setup

Same unit as cell 4: fork $x_0 \rightarrow x_1$, $x_0 \rightarrow x_2$ on the three-row source factorial, with asymmetric maps $x_1 := 2$ if $x_0 \neq 2$ else 0, and $x_2 := 2$ if $x_0 \neq 0$ else 0. For target x_2 , nonzero-positive labelling gives $E^+ = \{2, 3\}$ and $E^- = \{1\}$; on the table, $x_2 \neq 0$ if and only if $x_0 \neq 0$. Background knowledge includes exact-value predicates for the parent x_0 and the sibling distractor x_1 ; the target x_2 is excluded. The sibling is correlated with x_2 via x_0 but imperfect as a separator (`x1_val_2` holds on both negative row 1 and positive row 2). The locked reference $\mathcal{H}_{x_2}^*$ is the two-clause parent expansion of $x_0 \neq 0$.

3.6.2 Intended hypothesis $\mathcal{H}_{x_2}^*$

```
x2(A) :- x0_val_1(A).
x2(A) :- x0_val_2(A).
```

3.6.3 What happened

This cell uses the ECAI published configuration on unit U4 with learning target x_2 . After the initial optimum there are two rote rules, one for each positive. Non-deterministic folding replaces each identifier by the corresponding parent value in a single step, and both folds entail $\langle E^+, E^- \rangle$ immediately. The learner returns a solved framework with full ASP coverage, no assumptions, and no citation of the sibling.

3.6.4 What the learner did

Learning begins from rote coverage of the two positives:

```
x2(A) :- A=2.  
x2(A) :- A=3.
```

The first fold chosen is $A = 2$ (where $x_0 = 1$), which replaces the identifier by `x0_val_1`:

```
x2(A) :- x0_val_1(A).
```

That body does not cover the negative ($x_0 = 0$), so entailment succeeds without assumption introduction. The second fold is $A = 3$ (where $x_0 = 2$), which likewise yields a safe unary parent rule:

```
x2(A) :- x0_val_2(A).
```

Learning then stops with **nothing to fold**. Although sibling predicates (`x1_val_2`, `x1_val_0`) are present in the background knowledge, neither appears in a target body. Body-scope variables of the target rules are $\{x_0\}$.

3.6.5 Match assessment

This is an exact match to $\mathcal{H}_{x_2}^*$.

The two unary parent rules are precisely the locked `val` expansion of “ $x_0 \neq 0$ ” under nonzero-positive labelling of x_2 . ASP coverage is complete, the solution uses no assumptions, and the sibling distractor is not cited.

Comparison with cell 4. On the same fork under ECAI, target x_1 (cell 4) recovered $\mathcal{H}_{x_1}^*$ as `x0_val_0` / `x0_val_1` (parent $x_0 \neq 2$), while this cell recovers $\mathcal{H}_{x_2}^*$ as `x0_val_1` / `x0_val_2` (parent $x_0 \neq 0$). The learning path is the same in structure: two one-step parent folds, immediate entailment, no assumptions, sibling left unused. The difference is only which nonzero condition the asymmetric map induces, and which child appears in BK as the imperfect sibling distractor. Together, the two ECAI U4 cells show clean parent recovery on both branches of the fork.

Short verdict. Exact match to $\mathcal{H}_{x_2}^*$: two unary parent rules, no assumptions, sibling unused, full coverage (mirror of cell 4 on the other child).

3.7 6. ECAI $\times$ m12_u5_chain_curated $\times$ $t = x_2$

Cell directory: M12x_ecai2024/cells/m12_u5_chain_curated__target-x2/

Runner outcome: solved

ASP coverage: positives 4/4, negatives 2/2

3.7.1 Setup

Unit U5 is the chain $x_0 \rightarrow x_1 \rightarrow x_2$ on the curated six-row pilot support: for each $x_0 \in \{0, 1, 2\}$, the pairs (x_0, x_0) and $(x_0, (x_0 + 1) \bmod 3)$, with $x_2 := \text{copy}(x_1)$. Nonzero-positive labelling gives $E^+ = \{2, 3, 4, 5\}$ and $E^- = \{1, 6\}$; on the table, $x_2 \neq 0$ if and only if $x_1 \in \{1, 2\}$. Background knowledge includes exact-value predicates for both ancestors x_0 and x_1 ; the target x_2 is excluded. The direct parent x_1 perfectly separates $E^\pm$; the grandparent x_0 is correlated but imperfect ($x_0 = 0$ and $x_0 = 2$ each contain one positive and one negative). The locked reference $\mathcal{H}_{x_2}^*$ prefers the parent copy expansion.

3.7.2 Intended hypothesis $\mathcal{H}_{x_2}^*$

```
x2(A) :- x1_val_1(A).
x2(A) :- x1_val_2(A).
```

3.7.3 What happened

This cell uses the ECAI published configuration on unit U5 with learning target x_2 . After the initial optimum there are four rote rules, one for each positive. Non-deterministic folding proceeds by x_0 -slices rather than by the parent x_1 : the complete $x_0 = 1$ slice becomes a pure unary grandparent rule, while the incomplete $x_0 = 0$ and $x_0 = 2$ slices force assumption introduction. Contrary learning then recovers $x_1 \neq 0$ as the condition that defeats those assumptions. The learner returns a solved framework with full ASP coverage, but the target bodies cite the imperfect ancestor rather than the preferred parent.

3.7.4 What the learner did

Learning begins from rote coverage of the four positives. The first fold chosen is $A = 2$ (where $x_0 = 0, x_1 = 1$), which replaces the identifier by `x0_val_0` alone:

```
x2(A) :- x0_val_0(A).
```

That body also covers the negative $A = 1$ (where $x_0 = 0, x_1 = 0$), so entailment fails and assumption introduction adds `alpha_1`:

```
x2(A) :- alpha_1(A), x0_val_0(A).
```

Next, $A = 3$ (where $x_0 = 1, x_1 = 1$) folds to `x0_val_1`. Under this support every row with $x_0 = 1$ is positive, so the unary grandparent rule entails immediately and needs no assumption; $A = 4$ is then deleted as subsumed:

```
x2(A) :- x0_val_1(A).
```

Finally $A = 5$ (where $x_0 = 2, x_1 = 2$) folds to `x0_val_2`, which would also cover the negative $A = 6$, so assumption introduction adds `alpha_2`:

```
x2(A) :- alpha_2(A), x0_val_2(A).
```

Contrary folding for `c_alpha_1` and `c_alpha_2` first tries the matching x_0 literal (rejected: cannot introduce a further assumption on a body already tied to that assumption), then succeeds with

the parent literal `x1_val_0` on both negatives:

```
c_alpha_1(A) :- x1_val_0(A).
c_alpha_2(A) :- x1_val_0(A).
```

In the ASP solution the assumptions are realised by

```
alpha_1(A) :- not c_alpha_1(A), x0_val_0(A).
alpha_2(A) :- not c_alpha_2(A), x0_val_2(A).
```

so on the incomplete x_0 slices the assumption holds precisely when $x_1 \neq 0$. Body-scope variables of the target rules are $\{x_0\}$; the preferred parent x_1 appears only in the contrary conditions.

3.7.5 Match assessment

This is not an exact match to $\mathcal{H}_{x_2}^*$. The locked hypothesis is the unary parent expansion of $x_1 \in \{1, 2\}$. The learner never emits `x1_val_1` or `x1_val_2` in a target body.

Extensionally the learned theory is an x_0 -first case analysis repaired by parent-valued contraries:

$$(x_0 = 0 \wedge x_1 \neq 0) \vee (x_0 = 1) \vee (x_0 = 2 \wedge x_1 \neq 0),$$

which coincides with $x_1 \neq 0$ on this curated table, so ASP coverage is complete. Intensionally this is the parent-vs-ancestor divergence the unit was designed to probe: the imperfect grandparent occupies the target bodies, and the true parent is recovered only as the contrary of the assumptions needed to repair overgeneral x_0 folds.

The pattern is closely related to ECAI on U1 and U3: non-deterministic folding commits early to a one-literal body on a variable that does not perfectly separate $E^\pm$ within its incomplete slices, then uses assumption/contrary machinery—here ultimately citing the preferred separator—to restore coverage. Relative to semantic success for U5, the framework covers the labelling but does not recover the intended parent-copy intension.

Short verdict. Not an exact match: grandparent x_0 in target bodies with assumptions on incomplete slices; preferred parent x_1 only in contraries; full coverage.

3.8 7. ECAI $\times$ m12_u6_g1_min_diff $\times$ $t = x_2$

Cell directory: M12x_eca2024/cells/m12_u6_g1_min_diff__target-x2/

Runner outcome: solved

ASP coverage: positives 4/4, negatives 5/5

3.8.1 Setup

Unit U6 is the four-node G1 graph with mechanisms $x_2 := \min(x_0, x_1)$ and $x_3 := x_1 - x_2$ on the nine-row source factorial. For target x_2 , nonzero-positive labelling gives $E^+ = \{5, 6, 8, 9\}$ and $E^- = \{1, 2, 3, 4, 7\}$; on the table, $x_2 \neq 0$ if and only if both parents are nonzero. Background knowledge includes exact-value predicates for both parents x_0, x_1 and the descendant distractor x_3 ; the target x_2 is excluded. The descendant is imperfect as a separator of $E^\pm$ ($x_3 \neq 0$ only on rows 2, 3, 6). The locked reference $\mathcal{H}_{x_2}^*$ is the same four-clause both-parents-nonzero expansion as on U2.

3.8.2 Intended hypothesis $\mathcal{H}_{x_2}^*$

```
x2(A) :- x0_val_1(A), x1_val_1(A).
x2(A) :- x0_val_1(A), x1_val_2(A).
x2(A) :- x0_val_2(A), x1_val_1(A).
x2(A) :- x0_val_2(A), x1_val_2(A).
```

3.8.3 What happened

This cell uses the ECAI published configuration on unit U6 with learning target x_2 . After the initial optimum there are four rote rules, one for each positive. Non-deterministic folding proceeds by x_0 -slices: one-literal folds on `x0_val_1` and `x0_val_2` overgeneralise within those slices (they would cover negatives with $x_1 = 0$), so assumption introduction repairs each slice. Contrary learning recovers $x_1 \neq 0$. The descendant x_3 is available in BK but is never cited. The learner returns a solved framework with full ASP coverage that is not an exact syntactic match to $\mathcal{H}_{x_2}^*$, but encodes the same both-parents-nonzero concept.

3.8.4 What the learner did

Learning begins from rote coverage of the four positives. The first fold chosen is $A = 5$ (where $x_0 = 1, x_1 = 1$), which replaces the identifier by `x0_val_1` alone:

```
x2(A) :- x0_val_1(A).
```

That body would also cover the negative $A = 4$ (where $x_0 = 1, x_1 = 0$), so entailment fails and assumption introduction adds `alpha_1`:

```
x2(A) :- alpha_1(A), x0_val_1(A).
```

Positive $A = 6$ is then deleted as subsumed. Next, $A = 8$ folds to `x0_val_2`, which would cover the negative $A = 7$, so assumption introduction adds `alpha_2`:

```
x2(A) :- alpha_2(A), x0_val_2(A).
```

Positive $A = 9$ is subsumed. Contrary folding for `c_alpha_1` and `c_alpha_2` first tries the matching x_0 literal (rejected), then succeeds with `x1_val_0` on the negatives $A = 4$ and $A = 7$:

```
c_alpha_1(A) :- x1_val_0(A).
c_alpha_2(A) :- x1_val_0(A).
```

In the ASP solution the assumptions are realised by

```
alpha_1(A) :- not c_alpha_1(A), x0_val_1(A).
alpha_2(A) :- not c_alpha_2(A), x0_val_2(A).
```

so on an $x_0 \in \{1, 2\}$ row the assumption holds precisely when $x_1 \neq 0$. Body-scope variables of the target rules are $\{x_0\}$; the second parent x_1 appears in the contrary conditions. Descendant predicates `x3_val_*` do not appear in any learned rule.

3.8.5 Match assessment

This is not an exact match to $\mathcal{H}_{x_2}^*$. The learner does not produce the four ordinary both-parent conjunctions.

Judged against the semantic success description for U6 x_2 , however, the learned framework is a strong assumption-mediated encoding of both-parents-nonzero, and it avoids the descendant failure mode: x_3 is never cited. On an $x_0 \in \{1, 2\}$ row, x_2 holds if and only if the assumption is undefeated if and only if $x_1 \neq 0$. That is exactly the nonzero-positive semantics of min. ASP coverage is complete for that reason.

Comparison with ECAI on U2 (cell 2). The learning path and final intension are essentially the same as on the three-node min collider: unary x_0 folds, assumptions on both nonzero x_0 slices, and $x1_val_0$ contraries. The new U6 pressure is the descendant distractor in BK; under ECAI that pressure is resisted here.

Short verdict. Not an exact match to the locked four-clause $\mathcal{H}_{x_2}^*$, but an elegant assumption-mediated both-parents-nonzero solution with descendant x_3 unused and full coverage.

3.9 8. ECAI $\times$ m12_u6_g1_min_diff $\times$ $t = x_3$

Cell directory: M12x_ecai2024/cells/m12_u6_g1_min_diff__target-x3/

Runner outcome: solved

ASP coverage: positives 3/3, negatives 6/6

3.9.1 Setup

Same unit as cells 7 and 16: G1 with $x_2 := \min(x_0, x_1)$ and $x_3 := x_1 - x_2$ on the nine-row factorial. For target x_3 , nonzero-positive labelling gives $E^+ = \{2, 3, 6\}$ and $E^- = \{1, 4, 5, 7, 8, 9\}$; on the table, $x_3 \neq 0$ if and only if $x_1 > x_2$. Background knowledge includes exact-value predicates for all non-targets x_0, x_1, x_2 ; the target x_3 is excluded. The preferred parents are $\{x_1, x_2\}$; x_0 is a non-parent ancestor via x_2 . Neither parent alone separates $E^\pm$. The locked reference $\mathcal{H}_{x_3}^*$ is the three-clause parent case split of $x_1 > x_2$.

3.9.2 Intended hypothesis $\mathcal{H}_{x_3}^*$

```
x3(A) :- x1_val_1(A), x2_val_0(A).
x3(A) :- x1_val_2(A), x2_val_0(A).
x3(A) :- x1_val_2(A), x2_val_1(A).
```

3.9.3 What happened

This cell uses the ECAI published configuration on unit U6 with learning target x_3 . After the initial optimum there are three rote rules, one for each positive. Non-deterministic folding selects unary x_0 bodies on the positive-bearing slices $x_0 = 0$ and $x_0 = 1$; both overgeneralise within those slices, so assumption introduction is required. Contrary learning then introduces x_1 -valued conditions. The preferred parent x_2 does not appear in any target body or contrary. The learner returns a solved framework with full ASP coverage that is not an exact match to $\mathcal{H}_{x_3}^*$ and that cites the non-parent ancestor in the target rules.

3.9.4 What the learner did

Learning begins from rote coverage of the three positives. The first fold chosen is $A = 2$ (where $x_0 = 0, x_1 = 1, x_2 = 0$), which replaces the identifier by `x0_val_0` alone:

```
x3(A) :- x0_val_0(A).
```

That body also covers the negative $A = 1$ (where $x_0 = 0, x_1 = 0$), so entailment fails and assumption introduction adds `alpha_1`:

```
x3(A) :- alpha_1(A), x0_val_0(A).
```

Positive $A = 3$ is then deleted as subsumed. Next, $A = 6$ (where $x_0 = 1, x_1 = 2, x_2 = 1$) folds to `x0_val_1`, which would cover negatives $A = 4$ and $A = 5$ in the same x_0 slice, so assumption introduction adds `alpha_2`:

```
x3(A) :- alpha_2(A), x0_val_1(A).
```

There is no positive in the $x_0 = 2$ slice, so no corresponding target rule is learned. Contrary folding yields:

```
c_alpha_1(A) :- x1_val_0(A).
c_alpha_2(A) :- x1_val_0(A).
c_alpha_2(A) :- x1_val_1(A).
```

In the ASP solution the assumptions are realised by

```
alpha_1(A) :- not c_alpha_1(A), x0_val_0(A).
alpha_2(A) :- not c_alpha_2(A), x0_val_1(A).
```

so on the $x_0 = 0$ slice the assumption holds when $x_1 \neq 0$, and on the $x_0 = 1$ slice when $x_1 = 2$. Body-scope variables of the target rules are $\{x_0\}$; parent x_1 appears only in the contrary conditions; preferred parent x_2 is unused.

3.9.5 Match assessment

This is not an exact match to $\mathcal{H}_{x_3}^*$. The locked hypothesis uses both parents x_1 and x_2 . The learner never emits an `x2_val_*` literal.

Extensionally the learned theory is

$$(x_0 = 0 \wedge x_1 \neq 0) \vee (x_0 = 1 \wedge x_1 = 2),$$

which coincides with $x_1 > x_2$ on this factorial table (the only positives are rows 2, 3, and 6), so ASP coverage is complete. Intensionally this is a non-parent-ancestor divergence: the target bodies are organised by x_0 , one preferred parent enters only through contraries, and the other preferred parent x_2 is absent. Relative to the semantic success description for U6 x_3 , the framework covers the labelling but does not recover the intended two-parent inequality intension.

The learning path is structurally the same as ECAI outcomes on U5 and on U6 x_2 : an early one-literal fold on a variable that does not perfectly separate $E^\pm$ within its incomplete slices, followed by assumption introduction and contrary repair that recovers only part of the preferred parent structure. In this cell that repair reaches x_1 in the contraries but never introduces x_2 .

Short verdict. Not an exact match: non-parent ancestor x_0 in target bodies with assumptions; x_1 only in contraries; preferred parent x_2 unused; full coverage.

3.10 9. ECAI $\times$ m12_u7_diamond_noisy $\times$ $t = x_3$

Cell directory: M12x_ecai2024/cells/m12_u7_diamond_noisy__target-x3/

Runner outcome: solved

ASP coverage: positives 3/3, negatives 0/3

3.10.1 Setup

Unit U7 is a diamond with curated support. For each root value $x_0 \in \{0, 1, 2\}$, set $x_1 := (x_0 + 1) \bmod 3$ and include two values of x_2 , both functions of the common cause: $x_2 := (x_0 + 2) \bmod 3$ (siblings differ) and $x_2 := (x_0 + 1) \bmod 3$ (siblings collapse, equal to x_1). The sink is $x_3 := |x_1 - x_2|$. The six-row table therefore places, for every x_0 , one positive and one negative under nonzero-positive labelling of x_3 . On this table, $x_3 \neq 0$ if and only if $x_1 \neq x_2$. The examples are $E^+ = \{1, 3, 5\}$ and $E^- = \{2, 4, 6\}$. Each positive/negative twin shares (x_0, x_1) and differs only in x_2 . No unary **val**-union over $\{x_0\}$, $\{x_1\}$, or $\{x_2\}$ alone separates $E^\pm$; the parent pair $\{x_1, x_2\}$ does. Background knowledge includes exact-value predicates for x_0 , x_1 , and x_2 ; the target x_3 is excluded. This arm uses the ECAI published configuration (see Batch A control flow). The locked reference $\mathcal{H}_{x_3}^*$ enumerates the three sibling pairs on which the siblings differ.

3.10.2 Intended hypothesis $\mathcal{H}_{x_3}^*$

```
x3(A) :- x1_val_1(A), x2_val_2(A).
x3(A) :- x1_val_2(A), x2_val_0(A).
x3(A) :- x1_val_0(A), x2_val_1(A).
```

3.10.3 What happened

After rote learning there are three non-intensional positives. Under ECAI cover-and-repair at token budget 1, every target fold accepts a unary **x0_val_*** body; each such body also covers that positive's twin negative, so **gen2** fails entailment and **asm_intro(relto)** patches the fold with a new assumption. Contrary folding then rejects reuse of that assumption under a further **x0** fold (K0), accepts an **x1_val_*** fold plus a further assumption, and closes the tower by reusing the outer assumptions (OK). Preferred parent x_2 is present in BK throughout but is never selected as a folding literal. The trace ends with **gen1: nothing to fold** and no **Increasing folding tokens to:** line. The runner reports **solved**. ASP brave coverage on **bk.sol.asp**, however, covers all three positives and **rejects none of the three negatives**. This is therefore an ECAI **covering failure**: procedural completion without separation of $\langle E^+, E^- \rangle$.

3.10.4 What the learner did

Rote positives:

```
ert: x3(A) <- [A=1]
ert: x3(A) <- [A=3]
ert: x3(A) <- [A=5]
```

Target stage. For $A = 1$ (row $(0, 1, 2, 1)$), one-token folding selects `x0_val_0`:

```
1: folding [A=1] with x0_val_0(...)
0: DONE
folding result: x3(A) <- [x0_val_0(A)]
gen2: extended ABA does not entail <E+,E-> - looking for assumption relative to
gen2: generating NEW assumption: alpha_1(A)
assumption introduction result: x3(A) <- [alpha_1(A),x0_val_0(A)]
ert: c_alpha_1(A) <- [A=2]
```

The same pattern yields `alpha_2` with `x0_val_1` (from $A = 3$) and `alpha_3` with `x0_val_2` (from $A = 5$). Accepted target rules:

```
x3(A) :- alpha_1(A), x0_val_0(A).
x3(A) :- alpha_2(A), x0_val_1(A).
x3(A) :- alpha_3(A), x0_val_2(A).
```

Contrary stage. Folding `c_alpha_1` on $A = 2$ first tries `x0_val_0` and reuse of `alpha_1` (KO), then succeeds with `x1_val_1` and a new assumption `alpha_4`. The other two first-level contraries likewise KO on `x0` and accept `x1` plus `alpha_5` / `alpha_6`. Second-level contrary folds then reuse the outer assumptions successfully (OK), e.g. `c_alpha_4(A) :- alpha_1(A), x0_val_0(A)`. Learning stops when no non-intensional rules remain.

In the ASP solution the assumption defaults are

```
alpha_1(A) :- not c_alpha_1(A), x0_val_0(A).
alpha_2(A) :- not c_alpha_2(A), x0_val_1(A).
alpha_3(A) :- not c_alpha_3(A), x0_val_2(A).
alpha_4(A) :- not c_alpha_4(A), x1_val_1(A).
alpha_5(A) :- not c_alpha_5(A), x1_val_2(A).
alpha_6(A) :- not c_alpha_6(A), x1_val_0(A).
```

with first-level contraries on x_1 and second-level contraries looping back through the outer α_i and x_0 . Body-scope variables of the target rules are $\{x_0\}$; framework-scope variables are $\{x_0, x_1\}$. No learned body cites x_2 .

Why x_2 never arises. As in the Batch A control flow: with tokens at 1, each fold commits to a single BK literal; here the first success on every target fold is `x0_val_*`. When that overgeneralises, `gen2` repairs **that** fold by assumption introduction rather than searching for `x2_val_*` or a two-literal parent–parent body. Assumption introduction succeeds; under brave mode `gen6` continues without a cautious entailment re-check. On the contrary stage, after `x0` is KO, the next successful alternative is `x1` plus a new assumption—again without needing x_2 . Because this cover-and-repair derivation reaches **nothing to fold**, `genT` never raises the token budget toward the ceiling of 10. Background `x2_val_*` facts remain unused folding material; x_2 is not tried and discarded on the accepted path.

3.10.5 Match assessment

This is not an exact match to $\mathcal{H}_{x_3}^*$. The locked hypothesis is the three-clause both-parent case split of $x_1 \neq x_2$. Target bodies cite only the non-parent common cause x_0 , gated by assumptions; parent x_1 appears only in contrary conditions; preferred parent x_2 is absent from the learned framework.

Extensionally the outcome is a covering failure, not merely an intensional mismatch. Because each twin pair shares (x_0, x_1) , any theory whose learned conditions live only in $\{x_0, x_1\}$ cannot

separate those twins. ASP brave coverage confirms this: positives 3/3, negatives 0/3. Runner **solved** must not be read as a covering solution of $\langle E^+, E^- \rangle$; it records only that ABA Learning emptied the non-intensional list and wrote an artefact.

Relative to U7’s design purpose—forcing both-parent recovery by making every unary **val**-union imperfect—this cell is a clear ECAI failure of the Batch A cover-and-repair regime: early commitment to x_0 , repair through x_1 -mediated assumptions, no pressure to reach x_2 , and a returned framework that does not reject the curated twin negatives.

Comparison with ECAI on U6 x_3 (cell 8). Cell 8 shows a related early commitment to ancestor x_0 with preferred parent x_2 unused, but there the repaired theory still separates $E^\pm$ with full ASP coverage. Cell 9 has the same structural learning path, yet on U7’s twin design that path is fatal: shared (x_0, x_1) makes the missing x_2 an extensional covering failure.

Comparison with AAMAS on the same cell (cell 18). Deferred in full until cell 18 is written. Preview from the AAMAS artefact: greedy/**mgr** emits three-literal bodies citing x_0 , x_1 , and x_2 on each positive row—so both preferred parents appear, with non-parent ancestor residue—rather than an assumption tower that omits x_2 .

Short verdict. Covering failure despite runner **solved**: target bodies only on ancestor x_0 with a nested assumption tower over $\{x_0, x_1\}$; preferred parent x_2 never selected; ASP rejects 0/3 negatives.

4 Batch B — AAMAS (M12x_aamas2025)

4.1 AAMAS folding control flow

Batch B uses `configs/aamas2025_config.pl`: `learning_mode(brave)`, `folding_mode(greedy)`, `folding_selection(mgr)`, `folding_space(bk)`, and `asm_intro(relto)`. The shared **gen** skeleton still applies; what follows is how folding and selection specialise that skeleton on this arm.

Which rule is folded. Under `folding_selection(mgr)`, the engine does not pick an arbitrary non-intensional rule. After rote learning (and after any later contrary rote steps), it first computes greedy generalisations of the new non-intensional facts (**gen**: `computing greedy foldings`) and stores them in a most-general-first utility table. **gen1** then selects a rule among those **most general** candidates. The effect in M12x traces is that each positive’s specialised body is prepared up front, then applied one rote rule at a time.

How a fold is computed. Under `folding_mode(greedy)`, a single fold call walks the current body and, for each foldable literal, tries every eligible folding rule from the folding table, accumulating successful head replacements. There is **no token budget** and no **genT** iterative deepening. On a typical M12x positive $A = i$, the result is a **multi-literal** body that cites every background **val** predicate true of that row—for example

```
folding result: x3(A) <- [x0_val_0(A),x1_val_1(A),x2_val_0(A)]
```

after successive folds with each matching BK rule. The greedy procedure is committed (!): it does not leave alternative one-literal folds as open choice points in the way **nd** does.

`folding_space(bk)`. Only rules from the original background-knowledge block are eligible as folding material (rule ids before the learned-rule boundary). Learned target or contrary rules are not used as fold sources. Combined with greedy accumulation, this tends to specialise each positive to the full observed feature conjunction available in BK for that sample.

What happens when a fold overgeneralises. The shared **gen2** repair path (assumption introduction under **relto**, then **gen6** contrary rote learning) remains available. In practice on the M12x AAMAS cells inspected so far, greedy specialisation is usually already tight enough that the folded rule entails $\langle E^+, E^- \rangle$ immediately. The trace then reports **extended ABA entails <E+,E->** - using **folding** and returns to **gen1 without** introducing assumptions. When a greedy fold still overgeneralises, cover-and-repair can still fire—but assumption towers are the exception on this arm, not the default.

Contrast with ECAI (Batch A). ECAI typically proposes a one-literal fold at token budget 1 and, if that overgeneralises, repairs with assumptions. AAMAS typically proposes a fully specialised BK conjunction in one greedy step and keeps it when it already separates the examples. The shared failure modes therefore differ in shape: ECAI’s premature commitment is to an early unary column plus exception structure; AAMAS’s characteristic residue is **over-specialisation**—retaining extra BK conjuncts (siblings, ancestors, descendants) beside the literals that would constitute $\mathcal{H}_t^*$.

How to read AAMAS traces in this batch. An initial block **gen:** computing greedy foldings followed by per-sample folding **[A=i]** with ... / **DONE** lines is the precomputation of greedy generalisations. Later **begin greedy folding** under **gen1** applies that specialisation to the selected rote rule. **using folding** without intervening **generating NEW assumption** means the fold was accepted assumption-free. As in the shared skeleton, **nothing to fold** and **runner solved** record procedural completion; ASP coverage must still be checked separately.

4.2 10. AAMAS $\times$ m12_u1_separator_copy $\times$ $t = x_2$

Cell directory: M12x_aamas2025/cells/m12_u1_separator_copy__target-x2/

Runner outcome: solved

ASP coverage: positives 6/6, negatives 3/3

4.2.1 Setup

Same unit and target as cell 1: separator $x_1 \rightarrow x_2$ with isolated x_0 ; $x_2 := \text{copy}(x_1)$ on the nine-row factorial; nonzero-positive examples $E^+ = \{2, 3, 5, 6, 8, 9\}$ and $E^- = \{1, 4, 7\}$; exact-value background knowledge for x_0 and x_1 . This arm uses the AAMAS published configuration (**folding_mode(greedy)**, **folding_selection(mgr)**, **folding_space(bk)**, **asm_intro(relto)**).

4.2.2 Intended hypothesis $\mathcal{H}_{x_2}^*$

```
x2(A) :- x1_val_1(A).
x2(A) :- x1_val_2(A).
```

4.2.3 What happened

This cell uses the AAMAS published configuration on the same U1 instance. The learner returned a solved framework with full ASP coverage and with no assumptions or contraries. The learned target theory is a six-rule case split: each positive row becomes a conjunction of that row’s x_0 value and x_1 value. The parent x_1 appears in every body, but so does the isolated distractor x_0 . The result is therefore not an exact match to $\mathcal{H}_{x_2}^*$.

4.2.4 What the learner did

Learning again begins from rote rules for the six positives. Under greedy folding with selection `mgr` and folding space restricted to background knowledge, each positive is folded by adjoining both column values present on that row. For example, sample $A = 2$ has $x_0 = 0$ and $x_1 = 1$, and folds to:

```
x2(A) :- x0_val_0(A), x1_val_1(A).
```

Each such two-literal body entails the labelled example set immediately, so no assumption introduction is triggered. The same pattern is applied independently to every positive. The accepted target rules are:

```
x2(A) :- x0_val_0(A), x1_val_1(A).
x2(A) :- x0_val_0(A), x1_val_2(A).
x2(A) :- x0_val_1(A), x1_val_1(A).
x2(A) :- x0_val_1(A), x1_val_2(A).
x2(A) :- x0_val_2(A), x1_val_1(A).
x2(A) :- x0_val_2(A), x1_val_2(A).
```

There is no further generalisation that drops the x_0 literals and merges the six clauses into the two unary parent rules of $\mathcal{H}_{x_2}^*$. Body-scope variables are $\{x_0, x_1\}$. The solution uses no assumptions and no contraries.

4.2.5 Match assessment

The learned target rules are not an exact match to $\mathcal{H}_{x_2}^*$.

Extensionally, under mutual exclusivity and exhaustiveness of the three x_0 value predicates, the six conjunctions cover exactly the rows where $x_1 \in \{1, 2\}$. That is the same labelling as the intended two-rule parent hypothesis, which is why ASP coverage is complete.

Intensionally the solution is a positive-row case split that retains the isolated distractor in every body. The true parent is present, but the theory is a parent-plus-distractor superset of $\mathcal{H}_{x_2}^*$, not the compact unary parent rules. Relative to the ECAI twin on this unit, AAMAS avoids assumption-mediated repair and keeps x_1 in the target bodies, but it still fails to eliminate x_0 .

Short verdict. The cell is solved and extensionally adequate, with no assumptions, but intensionally it enumerates six specialised conjunctions that keep the isolated distractor beside the true parent.

4.3 11. AAMAS $\times$ m12_u2_collider_min $\times$ $t = x_2$

Cell directory: M12x_aamas2025/cells/m12_u2_collider_min__target-x2/

Runner outcome: solved

ASP coverage: positives 4/4, negatives 5/5

4.3.1 Setup

Same unit and target as cell 2: collider $x_0 \rightarrow x_2 \leftarrow x_1$ with $x_2 := \min(x_0, x_1)$ on the nine-row factorial; nonzero-positive examples $E^+ = \{5, 6, 8, 9\}$ and $E^- = \{1, 2, 3, 4, 7\}$; exact-value

background knowledge for both parents. This arm uses the AAMAS published configuration (`folding_mode(greedy)`, `folding_selection(mgr)`, `folding_space(bk)`, `asm_intro(relto)`).

4.3.2 Intended hypothesis $\mathcal{H}_{x_2}^*$

```
x2(A) :- x0_val_1(A), x1_val_1(A).
x2(A) :- x0_val_1(A), x1_val_2(A).
x2(A) :- x0_val_2(A), x1_val_1(A).
x2(A) :- x0_val_2(A), x1_val_2(A).
```

4.3.3 What happened

This cell uses the AAMAS published configuration on the same U2 instance. After the initial optimum, the engine has four rote rules covering the positives by sample identifier:

```
x2(A) :- A=5.
x2(A) :- A=6.
x2(A) :- A=8.
x2(A) :- A=9.
```

The trace then computes a greedy folding plan for each of those four rules. For every positive, `greedy/mgr` first replaces the identifier by the x_0 value on that row, then extends the body with the x_1 value on the same row. The planned foldings are therefore the four nonzero parent pairs:

- $A = 5 \rightarrow x0_val_1, x1_val_1$
- $A = 6 \rightarrow x0_val_1, x1_val_2$
- $A = 8 \rightarrow x0_val_2, x1_val_1$
- $A = 9 \rightarrow x0_val_2, x1_val_2$

Those foldings are then applied one at a time. After each application, entailment is checked against the **whole current theory**, not against the newly folded clause alone. So after the first fold, the theory still contains the remaining rote rules for $A = 6, 8, 9$; after the second, the remaining rotes for $A = 8, 9$; and so on. Each intermediate theory already separates $E^\pm$, so the trace reports **extended ABA entails $\langle E^+, E^- \rangle$ - using folding** and never introduces assumptions. Learning stops with **nothing to fold**. The final solution is the four specialised conjunctions, with no assumptions and no contraries, and with full ASP coverage.

4.3.4 What the learner did

The accepted target rules are:

```
x2(A) :- x0_val_1(A), x1_val_1(A).    % from A=5
x2(A) :- x0_val_1(A), x1_val_2(A).    % from A=6
x2(A) :- x0_val_2(A), x1_val_1(A).    % from A=8
x2(A) :- x0_val_2(A), x1_val_2(A).    % from A=9
```

Body-scope variables are $\{x_0, x_1\}$.

4.3.5 Match assessment

This is an exact match to $\mathcal{H}_{x_2}^*$.

The four conjunctions are precisely the locked `val` expansion of “both parents are nonzero.” ASP coverage is complete, and the solution uses no assumptions.

Contrast with ECAI on the same unit. Under ECAI (cell 2), non-deterministic folding first produces one-literal bodies such as `x2(A) :- x0_val_1(A)`, which overgeneralise within an x_0 -slice and force assumption introduction; the second parent is recovered only through contraries, yielding a compact assumption-mediated encoding of both-parents-nonzero that is semantically successful but not an exact syntactic match to $\mathcal{H}_{x_2}^*$. Under AAMAS, greedy folding specialises each positive to the full parent pair immediately, so each intermediate theory remains safe without assumptions and the final rules coincide with $\mathcal{H}_{x_2}^*$. Both arms recover the intended concept; they differ in presentation: ECAI’s assumption form versus AAMAS’s exact case enumeration.

Short verdict. Exact match to $\mathcal{H}_{x_2}^*$: four both-parent conjunctions, no assumptions, full coverage.

4.4 12. AAMAS $\times$ m12_u3_collider_max $\times$ $t = x_2$

Cell directory: M12x_aamas2025/cells/m12_u3_collider_max__target-x2/

Runner outcome: solved

ASP coverage: positives 8/8, negatives 1/1

4.4.1 Setup

Same unit and target as cell 3: collider $x_0 \rightarrow x_2 \leftarrow x_1$ with $x_2 := \max(x_0, x_1)$ on the nine-row factorial; under nonzero-positive labelling, $x_2 \neq 0$ if and only if at least one parent is nonzero. The positive examples are rows $\{2, 3, 4, 5, 6, 7, 8, 9\}$ and the sole negative is row $\{1\}$. Background knowledge contains exact-value predicates for both parents; the target x_2 is excluded. This arm uses the AAMAS published configuration (`folding_mode(greedy)`, `folding_selection(mgr)`, `folding_space(bk)`, `asm_intro(relto)`). The locked reference $\mathcal{H}_{x_2}^*$ is the four-clause `val` expansion of “at least one parent is nonzero.”

4.4.2 Intended hypothesis $\mathcal{H}_{x_2}^*$

```
x2(A) :- x0_val_1(A).
x2(A) :- x0_val_2(A).
x2(A) :- x1_val_1(A).
x2(A) :- x1_val_2(A).
```

4.4.3 What happened

This cell uses the AAMAS published configuration on the same U3 instance. After the initial optimum, the engine has eight rote rules covering the positives by sample identifier. The trace then computes a greedy folding plan for each of those rules. For every positive, `greedy/mgr` first replaces the identifier by the x_0 value on that row, then extends the body with the x_1 value on the same row. The planned foldings are therefore the eight nonzero parent pairs of the factorial (every cell except $(x_0, x_1) = (0, 0)$):

- $A = 2 \rightarrow \text{x0_val_0}, \text{x1_val_1}$
- $A = 3 \rightarrow \text{x0_val_0}, \text{x1_val_2}$
- $A = 4 \rightarrow \text{x0_val_1}, \text{x1_val_0}$

- $A = 5 \rightarrow x0_val_1, x1_val_1$
- $A = 6 \rightarrow x0_val_1, x1_val_2$
- $A = 7 \rightarrow x0_val_2, x1_val_0$
- $A = 8 \rightarrow x0_val_2, x1_val_1$
- $A = 9 \rightarrow x0_val_2, x1_val_2$

Those foldings are then applied one at a time. After each application, entailment is checked against the **whole current theory**, not against the newly folded clause alone. Each pairwise body matches only its own factorial cell, so it never covers the negative $A = 1$. Every intermediate theory already separates $E^\pm$, the trace reports **extended ABA entails <E+,E-> - using folding**, and no assumptions are introduced. Learning stops with **nothing to fold**. The final solution is the eight specialised conjunctions, with no assumptions and no contraries, and with full ASP coverage.

4.4.4 What the learner did

The accepted target rules are:

```
x2(A) :- x0_val_0(A), x1_val_1(A).
x2(A) :- x0_val_0(A), x1_val_2(A).
x2(A) :- x0_val_1(A), x1_val_0(A).
x2(A) :- x0_val_1(A), x1_val_1(A).
x2(A) :- x0_val_1(A), x1_val_2(A).
x2(A) :- x0_val_2(A), x1_val_0(A).
x2(A) :- x0_val_2(A), x1_val_1(A).
x2(A) :- x0_val_2(A), x1_val_2(A).
```

Body-scope variables are $\{x_0, x_1\}$.

4.4.5 Match assessment

This is not an exact match to $\mathcal{H}_{x_2}^*$. The locked hypothesis is the compact four-rule OR of unary nonzero conditions on either parent. The learner instead emits an exhaustive enumeration of the eight positive cells.

Extensionally the two presentations coincide: the eight conjunctions cover precisely those rows where $\max(x_0, x_1) \neq 0$, i.e. at least one parent is nonzero. ASP coverage is complete for that reason. Intensionally, however, the solution never discovers a one-literal generalisation such as $x2(A) :- x0_val_1(A)$; under greedy/**mgr** each positive is specialised to the full parent pair immediately, and that pair already entails safely, so there is no pressure to fold further or to introduce assumptions.

Contrast with ECAI on the same unit. Under ECAI (cell 3), non-deterministic folding produces an x_0 -first case analysis: pure unary rules on the complete $x_0 \in \{1, 2\}$ slices, and an assumption-mediated rule only on the incomplete $x_0 = 0$ slice. That is more compact than eight pairwise clauses and closer in spirit to a disjunctive reading, though still not the locked four-rule OR. Under AAMAS, the same unit yields the exhaustive positive-cell enumeration with both parents in every body and no assumptions. Both arms recover the intended concept; they differ in presentation: ECAI's mixed unary/assumption case analysis versus AAMAS's eight specialised conjunctions.

Contrast with AAMAS on U2. The same greedy pairwise specialisation matched $\mathcal{H}^*$ exactly on U2, because the locked min hypothesis *is* a set of both-parent conjunctions. On U3 the locked max hypothesis is a unary OR, so the same folding style recovers the concept extensionally but

misses $\mathcal{H}_{x_2}^*$ syntactically.

Short verdict. Not an exact match to the locked four-rule OR: eight both-parent conjunctions enumerating the positive cells, no assumptions, full coverage of the intended “at least one parent nonzero” concept.

4.5 13. AAMAS $\times$ m12_u4_fork_asymmetric $\times$ $t = x_1$

Cell directory: M12x_aamas2025/cells/m12_u4_fork_asymmetric__target-x1/

Runner outcome: solved

ASP coverage: positives 2/2, negatives 1/1

4.5.1 Setup

Same unit and target as cell 4: fork $x_0 \rightarrow x_1$, $x_0 \rightarrow x_2$ on the three-row source factorial; $x_1 := 2$ if $x_0 \neq 2$ else 0; nonzero-positive examples $E^+ = \{1, 2\}$ and $E^- = \{3\}$; on the table, $x_1 \neq 0$ if and only if $x_0 \in \{0, 1\}$. Background knowledge includes exact-value predicates for the parent x_0 and the sibling distractor x_2 . This arm uses the AAMAS published configuration (`folding_mode(greedy)`, `folding_selection(mgr)`, `folding_space(bk)`, `asm_intro(relto)`). The locked reference $\mathcal{H}_{x_1}^*$ is the two-clause unary parent expansion of $x_0 \neq 2$.

4.5.2 Intended hypothesis $\mathcal{H}_{x_1}^*$

```
x1(A) :- x0_val_0(A).
x1(A) :- x0_val_1(A).
```

4.5.3 What happened

This cell uses the AAMAS published configuration on the same U4 x_1 instance. After the initial optimum, the engine has two rote rules covering the positives by sample identifier. The trace then computes a greedy folding plan for each rule. For every positive, greedy/`mgr` first replaces the identifier by the x_0 value on that row, then extends the body with the sibling x_2 value on the same row. The planned foldings are therefore the observed parent–sibling pairs:

- $A = 1 \rightarrow \text{x0_val_0}, \text{x2_val_0}$
- $A = 2 \rightarrow \text{x0_val_1}, \text{x2_val_2}$

Those foldings are applied one at a time. Each pairwise body matches only its own row on this three-row table, so neither covers the negative $A = 3$. Every intermediate theory already separates $E^\pm$, the trace reports **extended ABA entails $\langle E^+, E^- \rangle$ - using folding**, and no assumptions are introduced. Learning stops with **nothing to fold**. The final solution cites both the true parent and the sibling distractor in every target body.

4.5.4 What the learner did

The accepted target rules are:

```
x1(A) :- x0_val_0(A), x2_val_0(A).
x1(A) :- x0_val_1(A), x2_val_2(A).
```

Body-scope variables are $\{x_0, x_2\}$.

4.5.5 Match assessment

This is not an exact match to $\mathcal{H}_{x_1}^*$. The locked hypothesis uses unary parent conditions only. The learner instead emits two specialised conjunctions that retain the sibling value observed on each positive row.

Extensionally the theory still separates $E^\pm$ with full ASP coverage: on this table the parent literals alone already suffice, and the sibling conjuncts are redundant specialisations. Intensionally this is a sibling-citation divergence. Relative to the unit’s semantic success description, the parent is recovered, but the imperfect sibling distractor is also cited—the failure mode the U4 design was meant to probe.

Comparison with ECAI on the same cell (cell 4). Under ECAI, non-deterministic folding stops after the one-literal parent folds `x0_val_0` and `x0_val_1`, which already entail safely, yielding an exact match to $\mathcal{H}_{x_1}^*$ with the sibling unused. Under AAMAS, greedy/`mgr` continues from the parent literal to the sibling literal on the same row before accepting the fold, so the sibling enters every body even though it is not needed for entailment. Both arms recover a covering theory centred on the parent; they differ in whether the correlated sibling is left out of the intension.

Short verdict. Not an exact match: parent recovered, but sibling x_2 cited in both bodies via greedy pairwise specialisation; full coverage, no assumptions.

4.6 14. AAMAS $\times$ m12_u4_fork_asymmetric $\times$ $t = x_2$

Cell directory: M12x_aamas2025/cells/m12_u4_fork_asymmetric__target-x2/

Runner outcome: solved

ASP coverage: positives 2/2, negatives 1/1

4.6.1 Setup

Same unit and target as cell 5: fork $x_0 \rightarrow x_1$, $x_0 \rightarrow x_2$ on the three-row source factorial; $x_2 := 2$ if $x_0 \neq 0$ else 0; nonzero-positive examples $E^+ = \{2, 3\}$ and $E^- = \{1\}$; on the table, $x_2 \neq 0$ if and only if $x_0 \neq 0$. Background knowledge includes exact-value predicates for the parent x_0 and the sibling distractor x_1 . This arm uses the AAMAS published configuration (`folding_mode(greedy)`, `folding_selection(mgr)`, `folding_space(bk)`, `asm_intro(relto)`). The locked reference $\mathcal{H}_{x_2}^*$ is the two-clause unary parent expansion of $x_0 \neq 0$.

4.6.2 Intended hypothesis $\mathcal{H}_{x_2}^*$

```
x2(A) :- x0_val_1(A).
x2(A) :- x0_val_2(A).
```

4.6.3 What happened

This cell uses the AAMAS published configuration on the same U4 x_2 instance. After the initial optimum, the engine has two rote rules covering the positives by sample identifier. The trace

then computes a greedy folding plan for each rule. For every positive, greedy/**mgr** first replaces the identifier by the x_0 value on that row, then extends the body with the sibling x_1 value on the same row. The planned foldings are therefore the observed parent–sibling pairs:

- $A = 2 \rightarrow \text{x0_val_1}, \text{x1_val_2}$
- $A = 3 \rightarrow \text{x0_val_2}, \text{x1_val_0}$

Those foldings are applied one at a time. Each pairwise body matches only its own row on this three-row table, so neither covers the negative $A = 1$. Every intermediate theory already separates $E^\pm$, the trace reports **extended ABA entails <E+,E-> - using folding**, and no assumptions are introduced. Learning stops with **nothing to fold**. The final solution cites both the true parent and the sibling distractor in every target body.

4.6.4 What the learner did

The accepted target rules are:

```
x2(A) :- x0_val_1(A), x1_val_2(A).
x2(A) :- x0_val_2(A), x1_val_0(A).
```

Body-scope variables are $\{x_0, x_1\}$.

4.6.5 Match assessment

This is not an exact match to $\mathcal{H}_{x_2}^*$. The locked hypothesis uses unary parent conditions only. The learner instead emits two specialised conjunctions that retain the sibling value observed on each positive row.

Extensionally the theory still separates $E^\pm$ with full ASP coverage: on this table the parent literals alone already suffice, and the sibling conjuncts are redundant specialisations. Intensionally this is a sibling-citation divergence of the same kind as on AAMAS x_1 .

Comparison with ECAI on the same cell (cell 5). Under ECAI, non-deterministic folding stops after the one-literal parent folds `x0_val_1` and `x0_val_2`, yielding an exact match to $\mathcal{H}_{x_2}^*$ with the sibling unused. Under AAMAS, greedy/**mgr** continues to the sibling literal on the same row, so x_1 enters every body even though it is not needed for entailment.

Comparison with AAMAS on x_1 (cell 13). The learning path is the same on both branches of the fork: parent first, then sibling, accept pairwise specialisations without assumptions. Cell 13 cites distractor x_2 beside parent x_0 ; this cell cites distractor x_1 beside parent x_0 . Together with the two exact ECAI U4 cells, the four U4 outcomes show a clean arm contrast: ECAI recovers unary parent $\mathcal{H}^*$ on both children; AAMAS recovers covering parent-centred rules that also cite the correlated sibling.

Short verdict. Not an exact match: parent recovered, but sibling x_1 cited in both bodies via greedy pairwise specialisation; full coverage, no assumptions (mirror of cell 13).

4.7 15. AAMAS $\times$ m12_u5_chain_curated $\times$ $t = x_2$

Cell directory: M12x_aamas2025/cells/m12_u5_chain_curated__target-x2/

Runner outcome: solved

ASP coverage: positives 4/4, negatives 2/2

4.7.1 Setup

Same unit and target as cell 6: chain $x_0 \rightarrow x_1 \rightarrow x_2$ on the curated six-row support with $x_2 := \text{copy}(x_1)$; nonzero-positive examples $E^+ = \{2, 3, 4, 5\}$ and $E^- = \{1, 6\}$; on the table, $x_2 \neq 0$ if and only if $x_1 \in \{1, 2\}$. Background knowledge includes exact-value predicates for both ancestors. This arm uses the AAMAS published configuration (`folding_mode(greedy)`, `folding_selection(mgr)`, `folding_space(bk)`, `asm_intro(relto)`). The locked reference $\mathcal{H}_{x_2}^*$ prefers the unary parent copy expansion.

4.7.2 Intended hypothesis $\mathcal{H}_{x_2}^*$

```
x2(A) :- x1_val_1(A).
x2(A) :- x1_val_2(A).
```

4.7.3 What happened

This cell uses the AAMAS published configuration on the same U5 instance. After the initial optimum, the engine has four rote rules covering the positives by sample identifier. The trace then computes a greedy folding plan for each rule. For every positive, greedy/`mgr` first replaces the identifier by the grandparent x_0 value on that row, then extends the body with the parent x_1 value on the same row. The planned foldings are therefore the four observed ancestor–parent pairs on the positives:

- $A = 2 \rightarrow x0_val_0, x1_val_1$
- $A = 3 \rightarrow x0_val_1, x1_val_1$
- $A = 4 \rightarrow x0_val_1, x1_val_2$
- $A = 5 \rightarrow x0_val_2, x1_val_2$

Those foldings are applied one at a time. Each pairwise body matches only its own row on this support, so none covers a negative. Every intermediate theory already separates $E^\pm$, the trace reports `extended ABA entails <E+,E-> - using folding`, and no assumptions are introduced. Learning stops with `nothing to fold`. The final solution cites both the preferred parent and the imperfect grandparent in every target body.

4.7.4 What the learner did

The accepted target rules are:

```
x2(A) :- x0_val_0(A), x1_val_1(A).
x2(A) :- x0_val_1(A), x1_val_1(A).
x2(A) :- x0_val_1(A), x1_val_2(A).
x2(A) :- x0_val_2(A), x1_val_2(A).
```

Body-scope variables are $\{x_0, x_1\}$.

4.7.5 Match assessment

This is not an exact match to $\mathcal{H}_{x_2}^*$. The locked hypothesis is the compact unary parent expansion. The learner instead emits four specialised conjunctions that retain the grandparent value observed on each positive row.

Extensionally the theory separates $E^\pm$ with full ASP coverage: each rule is a positive cell of the curated support, and together they cover exactly the rows where $x_1 \in \{1, 2\}$. Intensionally the parent x_1 is present in every body—so the preferred separator is recovered as a conjunct—but the imperfect ancestor x_0 is also cited throughout. That is the parent-vs-ancestor contamination the unit watches for: success is not the clean parent-only intension of $\mathcal{H}_{x_2}^*$.

Comparison with ECAI on the same cell (cell 6). Under ECAI, non-deterministic folding commits to unary x_0 bodies, forces assumptions on the incomplete x_0 slices, and recovers $x_1 \neq 0$ only through contraries; the preferred parent never appears in a target body. Under AAMAS, greedy specialisation keeps both ancestors in every body, needs no assumptions, and does place the parent in the target rules—but only beside the grandparent. Both arms cover the labelling and both miss exact $\mathcal{H}_{x_2}^*$; they differ in where the parent appears (contrary-only under ECAI versus redundant conjunct under AAMAS) and in whether assumptions are used.

Short verdict. Not an exact match: four parent–grandparent conjunctions; parent present but ancestor retained; no assumptions, full coverage.

4.8 16. AAMAS $\times$ m12_u6_g1_min_diff $\times$ $t = x_2$

Cell directory: M12x_aamas2025/cells/m12_u6_g1_min_diff__target-x2/

Runner outcome: solved

ASP coverage: positives 4/4, negatives 5/5

4.8.1 Setup

Same unit and target as cell 7: G1 with $x_2 := \min(x_0, x_1)$ and $x_3 := x_1 - x_2$ on the nine-row factorial; nonzero-positive examples $E^+ = \{5, 6, 8, 9\}$ and $E^- = \{1, 2, 3, 4, 7\}$; on the table, $x_2 \neq 0$ if and only if both parents are nonzero. Background knowledge includes both parents and the descendant distractor x_3 . This arm uses the AAMAS published configuration (`folding_mode(greedy)`, `folding_selection(mgr)`, `folding_space(bk)`, `asm_intro(relto)`). The locked reference $\mathcal{H}_{x_2}^*$ is the four-clause both-parents-nonzero expansion (same as U2); citing x_3 is an explicit failure mode.

4.8.2 Intended hypothesis $\mathcal{H}_{x_2}^*$

```
x2(A) :- x0_val_1(A), x1_val_1(A).
x2(A) :- x0_val_1(A), x1_val_2(A).
x2(A) :- x0_val_2(A), x1_val_1(A).
x2(A) :- x0_val_2(A), x1_val_2(A).
```

4.8.3 What happened

This cell uses the AAMAS published configuration on the same U6 x_2 instance. After the initial optimum, the engine has four rote rules covering the positives by sample identifier. The trace then computes a greedy folding plan for each rule. For every positive, `greedy/mgr` first folds in the x_0 value, then the x_1 value, then continues to the descendant x_3 value on the same row. The planned foldings are therefore three-literal specialisations:

- $A = 5 \rightarrow x0_val_1, x1_val_1, x3_val_0$
- $A = 6 \rightarrow x0_val_1, x1_val_2, x3_val_1$

- $A = 8 \rightarrow x0_val_2, x1_val_1, x3_val_0$
- $A = 9 \rightarrow x0_val_2, x1_val_2, x3_val_0$

Those foldings are applied one at a time. Each body matches only its own factorial cell, so none covers a negative. Every intermediate theory already separates $E^\pm$, the trace reports **extended ABA entails $\langle E^+, E^- \rangle$ - using folding**, and no assumptions are introduced. Learning stops with **nothing to fold**. The final solution contains the four parent pairs of $\mathcal{H}_{x_2}^*$, each contaminated by a descendant conjunct.

4.8.4 What the learner did

The accepted target rules are:

```
x2(A) :- x0_val_1(A), x1_val_1(A), x3_val_0(A).
x2(A) :- x0_val_1(A), x1_val_2(A), x3_val_1(A).
x2(A) :- x0_val_2(A), x1_val_1(A), x3_val_0(A).
x2(A) :- x0_val_2(A), x1_val_2(A), x3_val_0(A).
```

Body-scope variables are $\{x_0, x_1, x_3\}$.

4.8.5 Match assessment

This is not an exact match to $\mathcal{H}_{x_2}^*$. Stripping the `x3_val_*` literals would yield exactly the locked four-clause hypothesis; with those literals retained, every target body cites the descendant.

Extensionally the theory still separates $E^\pm$ with full ASP coverage: the parent pairs alone already characterise both-parents-nonzero, and the descendant conjuncts are redundant specialisations to the observed x_3 values on those four rows. Intensionally this is the explicit descendant failure mode U6 was designed to probe. Relative to semantic success, both parents are recovered, but the solution is not free of descendant citation.

Comparison with ECAI on the same cell (cell 7). Under ECAI, non-deterministic folding never cites x_3 ; it recovers both-parents-nonzero via assumptions and contraries. Under AAMAS, greedy specialisation recovers the parent pairs but continues folding into BK until the descendant is included. Both arms cover the labelling; they differ on whether the imperfect descendant enters the intension.

Comparison with AAMAS on U2 (cell 11). On U2 the same greedy pairwise style matched $\mathcal{H}^*$ exactly, because BK contained only the two parents. On U6 the extra descendant column in BK is enough for greedy/`mgr` to over-specialise beyond $\mathcal{H}_{x_2}^*$.

Short verdict. Not an exact match: four both-parent conjunctions each contaminated by descendant x_3 ; explicit descendant failure mode; full coverage, no assumptions.

4.9 17. AAMAS $\times$ m12_u6_g1_min_diff $\times$ $t = x_3$

Cell directory: M12x_aamas2025/cells/m12_u6_g1_min_diff__target-x3/

Runner outcome: solved

ASP coverage: positives 3/3, negatives 6/6

4.9.1 Setup

Same unit and target as cell 8: G1 with $x_2 := \min(x_0, x_1)$ and $x_3 := x_1 - x_2$ on the nine-row factorial; nonzero-positive examples $E^+ = \{2, 3, 6\}$ and $E^- = \{1, 4, 5, 7, 8, 9\}$; on the table, $x_3 \neq 0$ if and only if $x_1 > x_2$. Background knowledge includes exact-value predicates for x_0 , x_1 , and x_2 . This arm uses the AAMAS published configuration (`folding_mode(greedy)`, `folding_selection(mgr)`, `folding_space(bk)`, `asm_intro(relto)`). The locked reference $\mathcal{H}_{x_3}^*$ is the three-clause parent case split of $x_1 > x_2$; x_0 is a non-parent ancestor and should not appear in a preferred solution.

4.9.2 Intended hypothesis $\mathcal{H}_{x_3}^*$

```
x3(A) :- x1_val_1(A), x2_val_0(A).
x3(A) :- x1_val_2(A), x2_val_0(A).
x3(A) :- x1_val_2(A), x2_val_1(A).
```

4.9.3 What happened

This cell uses the AAMAS published configuration on the same U6 x_3 instance. After the initial optimum, the engine has three rote rules covering the positives by sample identifier. The trace then computes a greedy folding plan for each rule. For every positive, `greedy/mgr` folds in the x_0 , x_1 , and x_2 values observed on that row, yielding three-literal specialisations:

- $A = 2 \rightarrow x0_val_0, x1_val_1, x2_val_0$
- $A = 3 \rightarrow x0_val_0, x1_val_2, x2_val_0$
- $A = 6 \rightarrow x0_val_1, x1_val_2, x2_val_1$

Those foldings are applied one at a time. Each body matches only its own factorial cell, so none covers a negative. Every intermediate theory already separates $E^\pm$, the trace reports **extended ABA entails** `<E+,E-> - using folding`, and no assumptions are introduced. Learning stops with **nothing to fold**. The final solution contains the three parent pairs of $\mathcal{H}_{x_3}^*$, each extended by a non-parent ancestor conjunct.

4.9.4 What the learner did

The accepted target rules are:

```
x3(A) :- x0_val_0(A), x1_val_1(A), x2_val_0(A).
x3(A) :- x0_val_0(A), x1_val_2(A), x2_val_0(A).
x3(A) :- x0_val_1(A), x1_val_2(A), x2_val_1(A).
```

Body-scope variables are $\{x_0, x_1, x_2\}$.

4.9.5 Match assessment

This is not an exact match to $\mathcal{H}_{x_3}^*$. Stripping the `x0_val_*` literals would yield exactly the locked three-clause parent case split; with those literals retained, every target body cites the non-parent ancestor.

Extensionally the theory separates $E^\pm$ with full ASP coverage: the parent pairs alone already characterise $x_1 > x_2$ on this table, and the x_0 conjuncts are redundant specialisations to the

observed ancestor values on the three positive rows. Intensionally both preferred parents are present in every body—so the intended two-parent inequality structure is recovered as a sub-conjunction—but the solution is contaminated by ancestor citation. Relative to the semantic success description, this is therefore a partial recovery of the preferred intension rather than an exact match to $\mathcal{H}_{x_3}^*$.

Comparison with ECAI on the same cell (cell 8). Under ECAI, non-deterministic folding organises the target bodies by x_0 , recovers x_1 only through contraries, and never cites preferred parent x_2 . Under AAMAS, greedy specialisation places both x_1 and x_2 in every target body, but also retains x_0 . Both arms cover the labelling and both miss exact $\mathcal{H}_{x_3}^*$; they differ in whether the preferred two-parent structure appears in the target rules and in whether assumptions are used.

Comparison with AAMAS on U6 x_2 (cell 16). The same greedy over-specialisation pattern appears: the literals that would constitute $\mathcal{H}^*$ are recovered, then an additional BK column (x_3 there; x_0 here) is folded in as a redundant conjunct. In both cells the extra literal is the variable the unit treats as a structural distractor relative to the locked hypothesis.

Short verdict. Not an exact match: three parent-pair rules each contaminated by non-parent ancestor x_0 ; both preferred parents present; full coverage, no assumptions.

4.10 18. AAMAS $\times$ m12_u7_diamond_noisy $\times$ $t = x_3$

Cell directory: M12x_aamas2025/cells/m12_u7_diamond_noisy__target-x3/

Runner outcome: solved

ASP coverage: positives 3/3, negatives 3/3

4.10.1 Setup

Same unit and target as cell 9: diamond with curated support; $x_1 := (x_0 + 1) \bmod 3$; two x_2 arms per root ($x_2 := (x_0 + 2) \bmod 3$ and $x_2 := (x_0 + 1) \bmod 3$); sink $x_3 := |x_1 - x_2|$. Nonzero-positive examples $E^+ = \{1, 3, 5\}$ and $E^- = \{2, 4, 6\}$; on the table, $x_3 \neq 0$ if and only if $x_1 \neq x_2$. Each twin pair shares (x_0, x_1) and differs only in x_2 . Background knowledge includes exact-value predicates for x_0 , x_1 , and x_2 . This arm uses the AAMAS published configuration (see Batch B control flow). The locked reference $\mathcal{H}_{x_3}^*$ is the three-clause parent case split of $x_1 \neq x_2$; the common cause x_0 is a non-parent ancestor and should not appear in a preferred solution.

4.10.2 Intended hypothesis $\mathcal{H}_{x_3}^*$

```
x3(A) :- x1_val_1(A), x2_val_2(A).
x3(A) :- x1_val_2(A), x2_val_0(A).
x3(A) :- x1_val_0(A), x2_val_1(A).
```

4.10.3 What happened

After the initial optimum there are three rote rules, one per positive. The trace first computes greedy foldings for each (`gen: computing greedy foldings`). For every positive, greedy/`mgr` folds in the x_0 , x_1 , and x_2 values observed on that row, yielding three-literal specialisations:

- $A = 1 \rightarrow x0_val_0, x1_val_1, x2_val_2$

- $A = 3 \rightarrow x0_val_1, x1_val_2, x2_val_0$
- $A = 5 \rightarrow x0_val_2, x1_val_0, x2_val_1$

Those foldings are applied one at a time under **gen1**. Each body matches only its own positive cell (the twin negative shares (x_0, x_1) but not x_2), so none covers a negative. Every intermediate theory already separates $E^\pm$; the trace reports **extended ABA entails $\langle E+, E- \rangle$ - using folding**; no assumptions are introduced. Learning stops with **nothing to fold**. The final solution contains the three parent pairs of $\mathcal{H}_{x_3}^*$, each contaminated by a non-parent ancestor conjunct.

4.10.4 What the learner did

The accepted target rules are:

```
x3(A) :- x0_val_0(A), x1_val_1(A), x2_val_2(A).
x3(A) :- x0_val_1(A), x1_val_2(A), x2_val_0(A).
x3(A) :- x0_val_2(A), x1_val_0(A), x2_val_1(A).
```

Body-scope variables are $\{x_0, x_1, x_2\}$.

4.10.5 Match assessment

This is not an exact match to $\mathcal{H}_{x_3}^*$. Stripping the `x0_val_*` literals would yield exactly the locked three-clause parent case split; with those literals retained, every target body cites the non-parent ancestor.

Extensionally the theory separates $E^\pm$ with full ASP coverage: the parent pairs alone already characterise $x_1 \neq x_2$ on this table, and the x_0 conjuncts are redundant specialisations to the observed root values on the three positive rows. Intensionally both preferred parents are present in every body—so the intended both-parent inequality structure is recovered as a sub-conjunction—but the solution is contaminated by ancestor citation. Relative to U7’s design purpose, this is therefore a partial recovery of the preferred intension rather than an exact match to $\mathcal{H}_{x_3}^*$, and it is the Batch B over-specialisation pattern (see AAMAS folding control flow).

Comparison with ECAI on the same cell (cell 9). Under ECAI, cover-and-repair commits to unary x_0 bodies, builds a nested assumption tower over $\{x_0, x_1\}$, never selects x_2 , and fails ASP coverage of the negatives (0/3). Under AAMAS, greedy specialisation places both x_1 and x_2 in every target body, needs no assumptions, and fully separates $E^\pm$ —but also retains x_0 . The arms therefore diverge sharply on this unit: ECAI is a covering failure that misses a preferred parent; AAMAS is extensionally adequate and recovers both parents as conjuncts, missing exact $\mathcal{H}_{x_3}^*$ only through ancestor residue.

Comparison with AAMAS on U6 x_3 (cell 17). The same greedy pattern appears: the literals that would constitute $\mathcal{H}^*$ are recovered, then an additional BK column (ancestor x_0) is folded in as a redundant conjunct. In both cells the extra literal is the structural distractor relative to the locked hypothesis; on U7 that distractor is the common cause that the curated twins were designed to make imperfect as a unary separator.

Short verdict. Not an exact match: three parent-pair rules each contaminated by non-parent ancestor x_0 ; both preferred parents present; full coverage, no assumptions.